ASSIGNMENT-10.2

2303A51042

BATCH – 29

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:

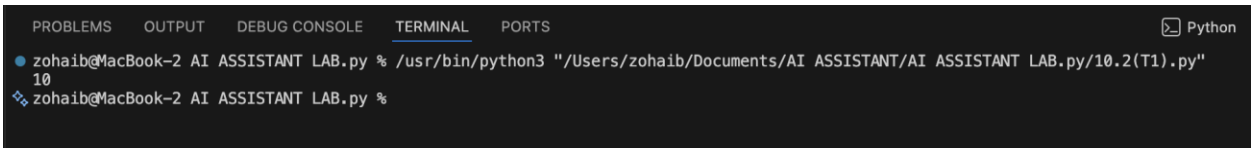
Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

CODE:

```python
def calculate_total(nums):
    total = 0
    for n in nums:
        total += n
    return total


# Example test
print(calculate_total([1, 2, 3, 4]))
```

OUTPUT:



OBSERVATION:

The AI successfully identified syntax errors such as missing colons and incorrect indentation, as well as logical errors like variable mismatch. After correction, the function executed properly and returned the expected output. This demonstrates effective automated error detection and correction.

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style

guidelines.

Sample Input Code:

def findSum(a,b):return a+b

print(findSum(5,10))

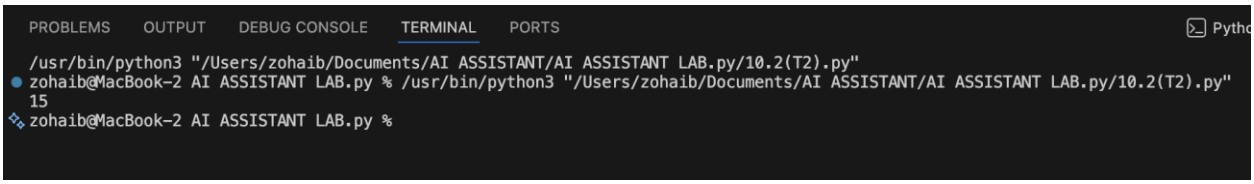
Expected Output-2:

Well-structured, consistently formatted Python code following standard

style conventions.

CODE:

OUTPUT:



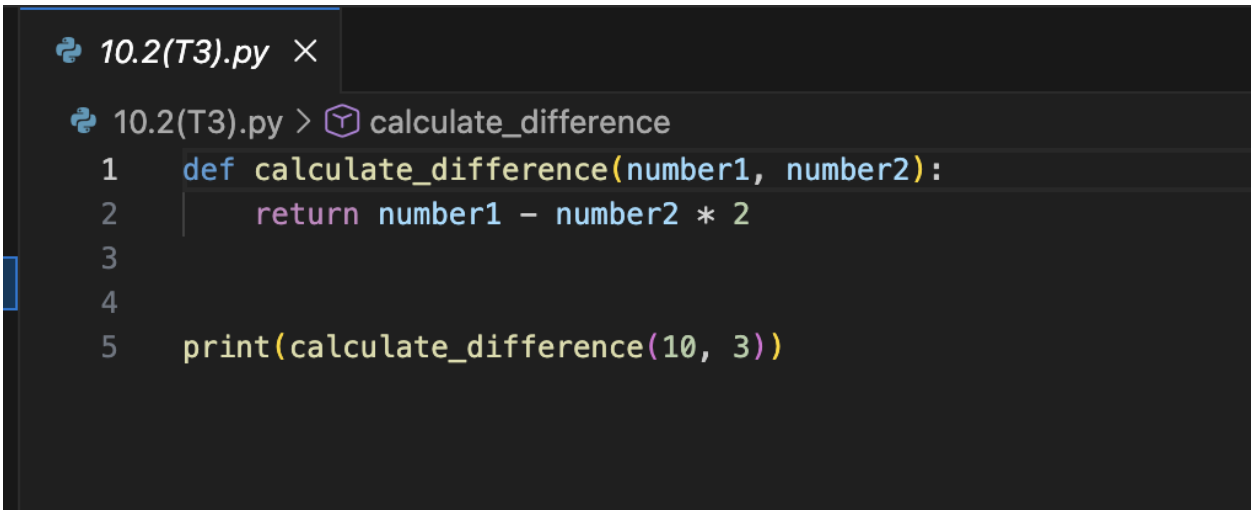
OBSERVATION:

The AI refactored the code to follow PEP 8 guidelines by improving naming conventions, spacing, and formatting. The updated code is more readable, structured, and maintainable while preserving original functionality..

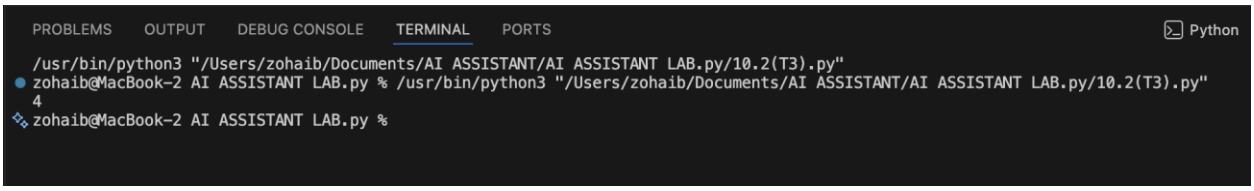
Task Description -3(Code Clarity Improvement)

Task:
Use AI to improve code readability without changing its functionality.
Sample Input Code:
def f(x,y):
return x-y*2
print(f(10,3))
Expected Output-3:
Python code rewritten with meaningful function and variable names,
proper indentation, and improved clarity.

CODE:



OUTPUT:



OBSERVATION:

The AI improved code readability by replacing unclear function and variable names with meaningful ones. The logic remained unchanged, but the code became easier to understand and maintain.

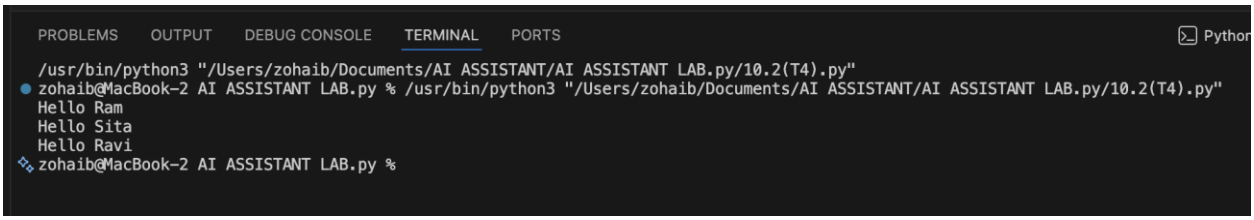
Task Description -4(Structural Refactoring)

Task:
Use AI to refactor repetitive code into reusable functions.
Sample Input Code:
print("Hello Ram")

print("Hello Sita")
print("Hello Ravi")
Expected Output-4:
Modular Python code using reusable functions to eliminate repetition.

CODE:

OUTPUT:



OBSERVATION:

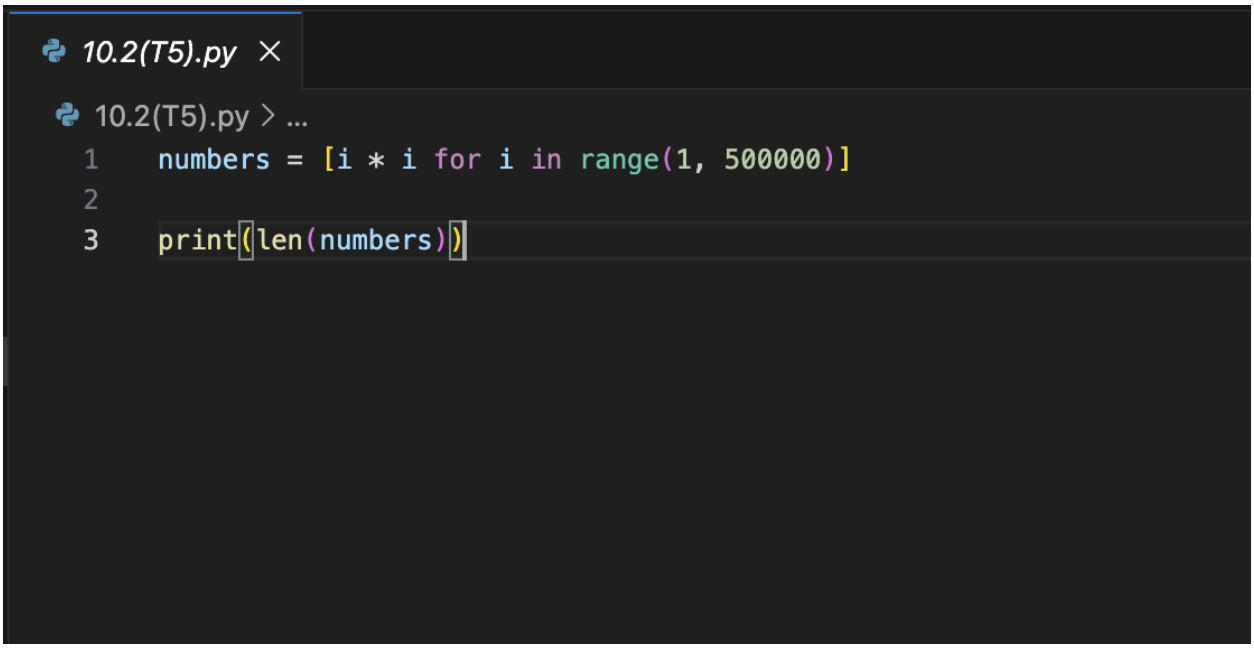
Repetitive code was refactored into a reusable function, improving modularity and reducing redundancy. The updated structure makes the program more scalable and easier to extend.

Task Description -5(Efficiency Enhancement)

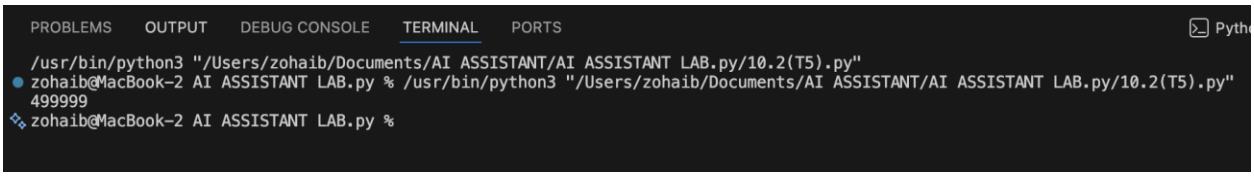
Task:
Use AI to optimize Python code for better performance.
Sample Input Code:

numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
Expected Output-5:
Optimized Python code that achieves the same result with improved performance.

CODE:



OUTPUT :



OBSERVATION :

The AI optimized the program by replacing a loop with list comprehension, improving execution speed and performance. The optimized code produced the same output more efficiently.